



Estudante: _____

1. Otimização de Código I (7pts)

```
1  g() {  
2  B0:  
3      x := 10  
4      y := 2  
5      i := 0  
6      total := 0  
7  
8  B1:  
9      limit := x + 30  
10     a := x * y  
11     b := x * y  
12     if (a > limit) goto B2  
13  
14     B3:  
15     c := a + 10  
16     j := 0  
17  
18     B4:  
19     k := x * 50  
20     c := c + k  
21     j := j + 1  
22     if (j < 10) goto B4 else goto B5  
23  
24     B2:  
25     c := b + 20  
26     d := x * y  
27     c := c + d  
28  
29     B5:  
30     total := total + c  
31     i := i + 1  
32     if (i < 5) goto B1  
33  
34     B6:  
35     return total  
36 }
```

- (a) O código acima está em forma SSA (*Static Single Assignment*)? Justifique. (1pt)
- (b) Desenhe o Grafo de Fluxo de Controle (CFG) do programa no nível de blocos básicos. (1pt)
- (c) Aplique as seguintes otimizações no código original: **Propagação de Constantes** e **Eliminação de Código Morto**. Mostre o código resultante e aponte as constantes propagada e o código morto eliminado. (3pts)
- (d) Se executarmos a técnica de Duplicação de Cauda no código original, o código resultante será mais eficiente ou apenas aumentará de tamanho? Justifique. (2pts)

2. Alocação de Registradores (7pts)

```
1 h() {
2   L0:
3     a := 5
4     b := read()
5
6   L1:
7     c := a + b
8     d := b * 2
9
10  L2:
11    e := c - d
12    f := a + e
13    d := d + 1
14    if (d < 20) goto L2
15
16  L3:
17    a := a - 1
18    if (a > 0) goto L1
19
20  L4:
21    print(f)
22    print(b)
23 }
```

- (a) Desenhe o Grafo de Fluxo de Controle (CFG) no nível de instrução. (1pt)
- (b) Execute a **Análise de Vida de Variáveis** e compute os conjuntos IN e OUT de cada instrução. (3pts)
- (c) Desenhe o Grafo de Interferência de Registradores (RIG). Qual o **número mínimo de registradores físicos necessários** para que o código execute sem a necessidade de salvar variáveis na memória? (2pts)
- (d) Explique por que a etapa de alocação de registradores em compiladores modernos não busca uma solução "absolutamente ótima" em todos os casos. (1pt)

3. Gerenciamento de Memória (4pts)

Os algoritmos Mark & Sweep (I) e Stop & Copy (II) possuem comportamentos distintos em relação ao uso do espaço de endereçamento e tempo de execução. Imagine um cenário onde o seu espaço disponível para alocação dinâmica está 95% ocupado por objetos vivos (ou seja, há pouco lixo a ser coletado), e esses objetos são de vida longa (permanecem vivos por quase toda a execução). Nesse cenário específico, qual dos dois algoritmos causaria um **impacto de desempenho negativo** maior no sistema? Justifique sua resposta.

4. Otimização de *Peephole* em MIPS (4pts)

O código abaixo foi gerado por um compilador sem otimizações para a arquitetura MIPS. Identifique pelo menos 4 oportunidades de otimização de *peephole* (instruções redundantes, simplificações algébricas ou saltos desnecessários) e escreva a versão otimizada.

```
1 lw    $t0, 0($a0)
2 move  $t1, $t0
3 addi  $t2, $t1, 0
4 mul   $t3, $t2, 2
5 sw    $t3, 4($a0)
6 beq   $zero, $zero, L1
7
8 L1:
9 add   $v0, $t3, $zero
10 jr   $ra
```

5. Otimização de Código II (3pts)

Sobre a realização de otimizações em diferentes estágios do processo de compilação (**Código Fonte, Representação Intermediária e Assembly**), assinale a alternativa que contém apenas as observações corretas:

I. Realizar otimizações diretamente no Código Fonte (AST) tem a vantagem de permitir transformações baseadas na semântica de alto nível da linguagem, como o alinhamento de funções e a reestruturação de laços complexos antes que a estrutura original se perca.

II. Uma desvantagem significativa de se limitar às otimizações no Código Fonte é a impossibilidade de prever com precisão o custo real de desempenho, visto que não há informações sobre o mapeamento de registradores ou latência de instruções.

III. A Representação Intermediária (IR) tem como principal vantagem a portabilidade e a neutralidade, sendo o estágio ideal para otimizações clássicas que podem ser reaproveitadas para múltiplos front-ends (linguagens) e back-ends (arquiteturas).

IV. Uma desvantagem das otimizações na Representação Intermediária (IR) é que, nesse estágio, o compilador já perdeu parte da expressividade semântica do código original e, ao mesmo tempo, ainda não consegue explorar instruções específicas do hardware (como instruções SIMD ou vetoriais).

V. Otimizações em Assembly permitem ajustes como o Agendamento de Instruções (*Instruction Scheduling*) e a alocação física de registradores baseada nas características reais da CPU.

VI. A maior desvantagem de realizar otimizações no nível de Assembly é a total falta de portabilidade, o que exige que o otimizador seja reescrito ou adaptado para cada nova arquitetura de processador suportada pelo compilador.

Estão corretas:

- (A) Apenas I, III e V.
- (B) Apenas II, IV e VI.
- (C) Apenas I, II, III e IV.
- (D) Todas as afirmações (I, II, III, IV, V e VI).
- (E) Nenhuma das alternativas anteriores.